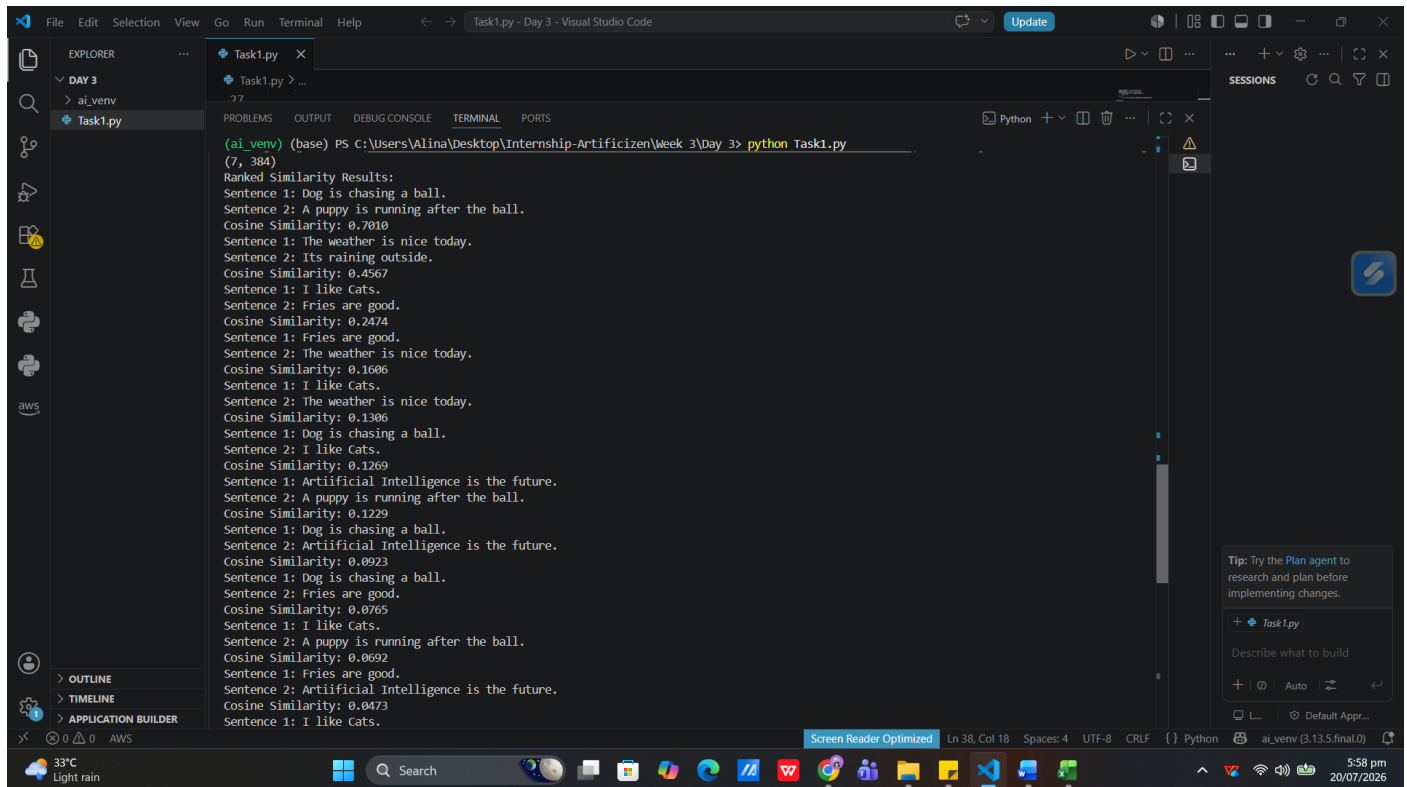
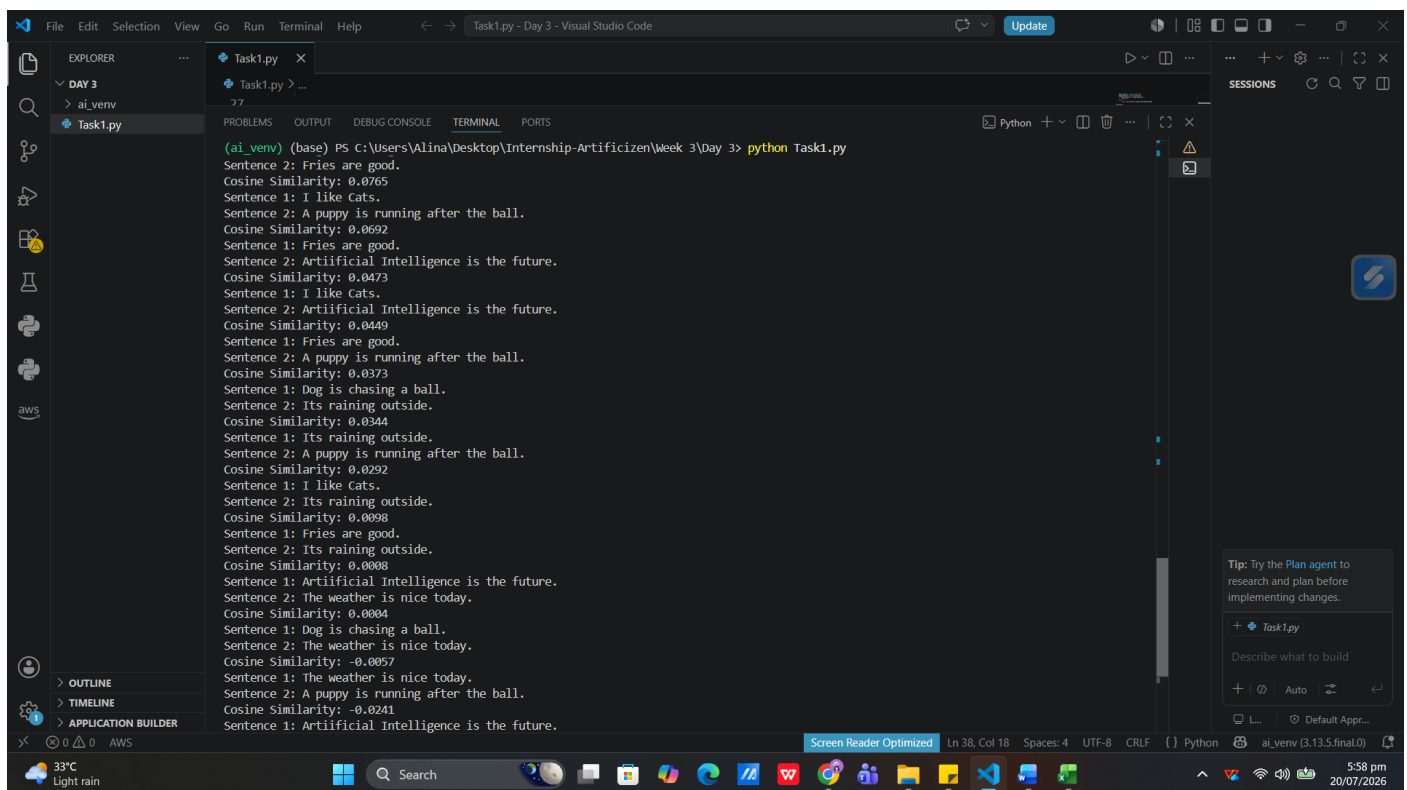


Practice Questions

1. Use sentence-transformers with all-MiniLM-L6-v2 to embed “A dog is chasing a ball” and five other sentences. Compute cosine similarity between all pairs using numpy and rank them from most to least similar.



```
(ai_venv) (base) PS C:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 3> python Task1.py
(7, 384)
Ranked Similarity Results:
Sentence 1: Dog is chasing a ball.
Sentence 2: A puppy is running after the ball.
Cosine Similarity: 0.7810
Sentence 1: The weather is nice today.
Sentence 2: Its raining outside.
Cosine Similarity: 0.4567
Sentence 1: I like Cats.
Sentence 2: Fries are good.
Cosine Similarity: 0.2474
Sentence 1: Fries are good.
Sentence 2: The weather is nice today.
Cosine Similarity: 0.1606
Sentence 1: I like Cats.
Sentence 2: The weather is nice today.
Cosine Similarity: 0.1306
Sentence 1: Dog is chasing a ball.
Sentence 2: I like Cats.
Cosine Similarity: 0.1269
Sentence 1: Artificial Intelligence is the future.
Sentence 2: A puppy is running after the ball.
Cosine Similarity: 0.1229
Sentence 1: Dog is chasing a ball.
Sentence 2: Artificial Intelligence is the future.
Cosine Similarity: 0.0923
Sentence 1: Dog is chasing a ball.
Sentence 2: Fries are good.
Cosine Similarity: 0.0765
Sentence 1: I like Cats.
Sentence 2: A puppy is running after the ball.
Cosine Similarity: 0.0692
Sentence 1: Fries are good.
Sentence 2: Artificial Intelligence is the future.
Cosine Similarity: 0.0473
Sentence 1: I like Cats.
```



```
(ai_venv) (base) PS C:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 3> python Task1.py
Sentence 2: Fries are good.
Cosine Similarity: 0.0765
Sentence 1: I like Cats.
Sentence 2: A puppy is running after the ball.
Cosine Similarity: 0.0692
Sentence 1: Fries are good.
Sentence 2: Artificial Intelligence is the future.
Cosine Similarity: 0.0473
Sentence 1: I like Cats.
Sentence 2: Artificial Intelligence is the future.
Cosine Similarity: 0.0449
Sentence 1: Fries are good.
Sentence 2: A puppy is running after the ball.
Cosine Similarity: 0.0373
Sentence 1: Dog is chasing a ball.
Sentence 2: Its raining outside.
Cosine Similarity: 0.0344
Sentence 1: Its raining outside.
Sentence 2: A puppy is running after the ball.
Cosine Similarity: 0.0292
Sentence 1: I like Cats.
Sentence 2: Its raining outside.
Cosine Similarity: 0.0098
Sentence 1: Fries are good.
Sentence 2: Its raining outside.
Cosine Similarity: 0.0008
Sentence 1: Artificial Intelligence is the future.
Sentence 2: The weather is nice today.
Cosine Similarity: 0.0004
Sentence 1: Dog is chasing a ball.
Sentence 2: The weather is nice today.
Cosine Similarity: -0.0057
Sentence 1: The weather is nice today.
Sentence 2: A puppy is running after the ball.
Cosine Similarity: -0.0241
Sentence 1: Artificial Intelligence is the future.
```

2. Write a function `semantic_search(query, documents)` that embeds the query, embeds all documents, and returns the top-3 most similar documents with their cosine similarity scores.

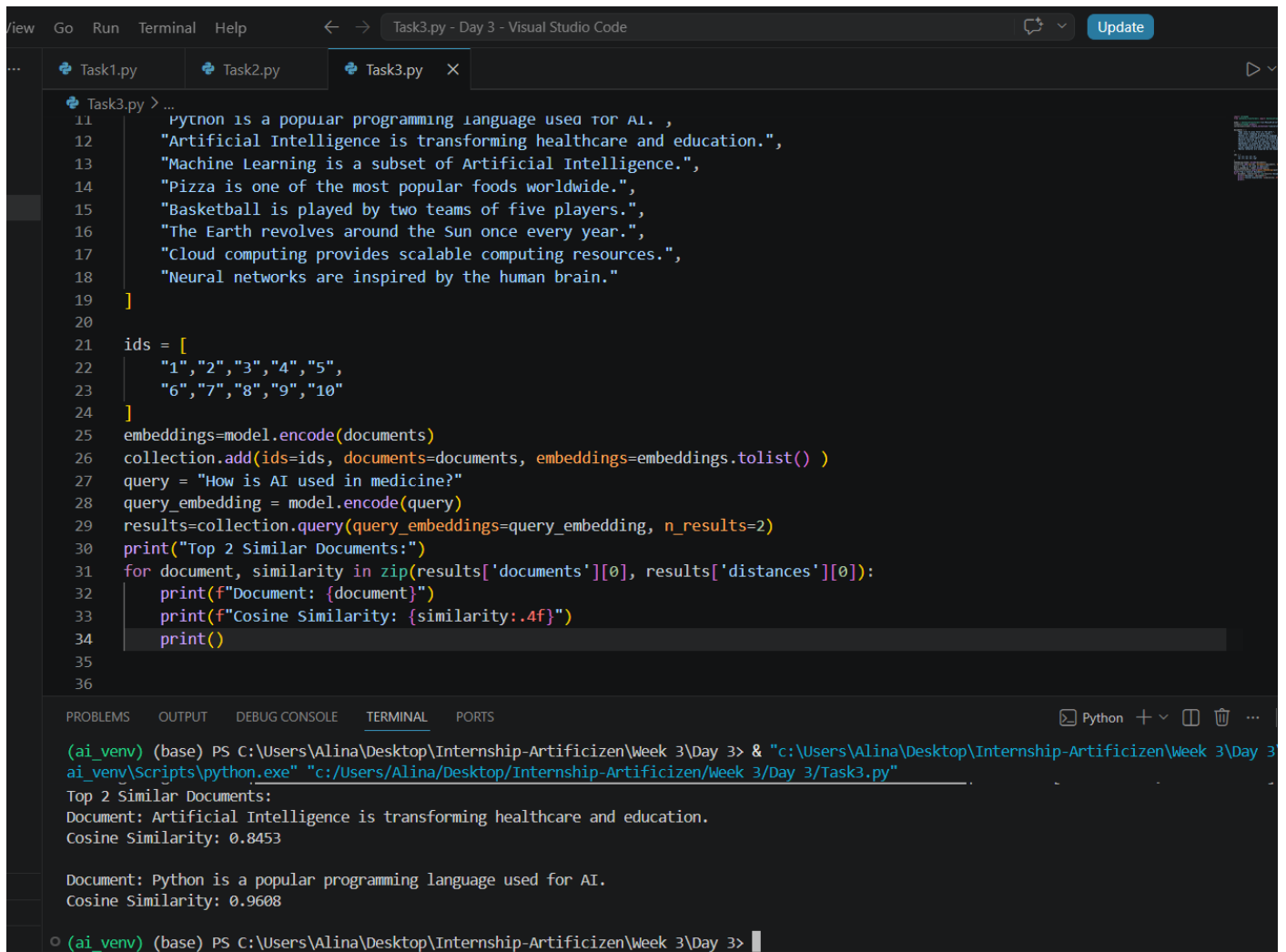
The screenshot displays a Visual Studio Code interface with two tabs at the top: "Task1.py" and "Task2.py". The active tab, "Task2.py", contains the following Python code:

```
16 def cosine_similarity(vec1, vec2):  
17     return np.dot(vec1, vec2) / (np.linalg.norm(vec1) * np.linalg.norm(vec2))  
18  
19 def semantic_search(query, documents):  
20     query_embedding=model.encode(query)  
21     document_embeddings=model.encode(documents)  
22     result=[]  
23     for i in range(len(documents)):  
24         similarity=cosine_similarity(query_embedding,document_embeddings[i])  
25         result.append((documents[i],similarity))  
26  
27     result.sort(key=lambda x:x[1],reverse=True)  
28     return result  
29  
30 query="The weather is cool today."  
31 top_results=semantic_search(query,documents)  
32 print("Top 3 Similar Documents:")  
33 for document, similarity in top_results[:3]:  
34     print(f"Document: {document}")  
35     print(f"Cosine Similarity: {similarity:.4f}")  
36     print()
```

Below the code editor is the TERMINAL panel, which shows the execution of the script. The command prompt indicates the current directory is C:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 3. The output matches the print statements in the code:

```
(ai_venv) (base) PS C:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 3> & "c:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 3\ai_venv\Scripts\python.exe" "c:/Users/Alina/Desktop/Internship-Artificizen/Week 3/Day 3/Task2.py"  
Top 3 Similar Documents:  
Document: The weather is nice today.  
Cosine Similarity: 0.8464  
  
Document: Its raining outside.  
Cosine Similarity: 0.4184  
  
Document: I like Cats.  
Cosine Similarity: 0.0643
```

3. Create a ChromaDB in-memory collection, add 10 short paragraphs from any topic, query it with a natural language question, and print the top-2 results.



The screenshot shows the Visual Studio Code editor with a file named 'Task3.py' open. The script defines a list of documents, encodes them into embeddings, and then queries a collection for the top 2 similar documents to a specific query. The terminal output shows the results of this query, displaying the top 2 similar documents and their cosine similarity scores.

```
11 python is a popular programming language used for AI. ,
12 "Artificial Intelligence is transforming healthcare and education.",
13 "Machine Learning is a subset of Artificial Intelligence.",
14 "Pizza is one of the most popular foods worldwide.",
15 "Basketball is played by two teams of five players.",
16 "The Earth revolves around the Sun once every year.",
17 "Cloud computing provides scalable computing resources.",
18 "Neural networks are inspired by the human brain."
19 ]
20
21 ids = [
22     "1","2","3","4","5",
23     "6","7","8","9","10"
24 ]
25 embeddings=model.encode(documents)
26 collection.add(ids=ids, documents=documents, embeddings=embeddings.tolist() )
27 query = "How is AI used in medicine?"
28 query_embedding = model.encode(query)
29 results=collection.query(query_embeddings=query_embedding, n_results=2)
30 print("Top 2 Similar Documents:")
31 for document, similarity in zip(results['documents'][0], results['distances'][0]):
32     print(f"Document: {document}")
33     print(f"Cosine Similarity: {similarity:.4f}")
34     print()
35
36
```

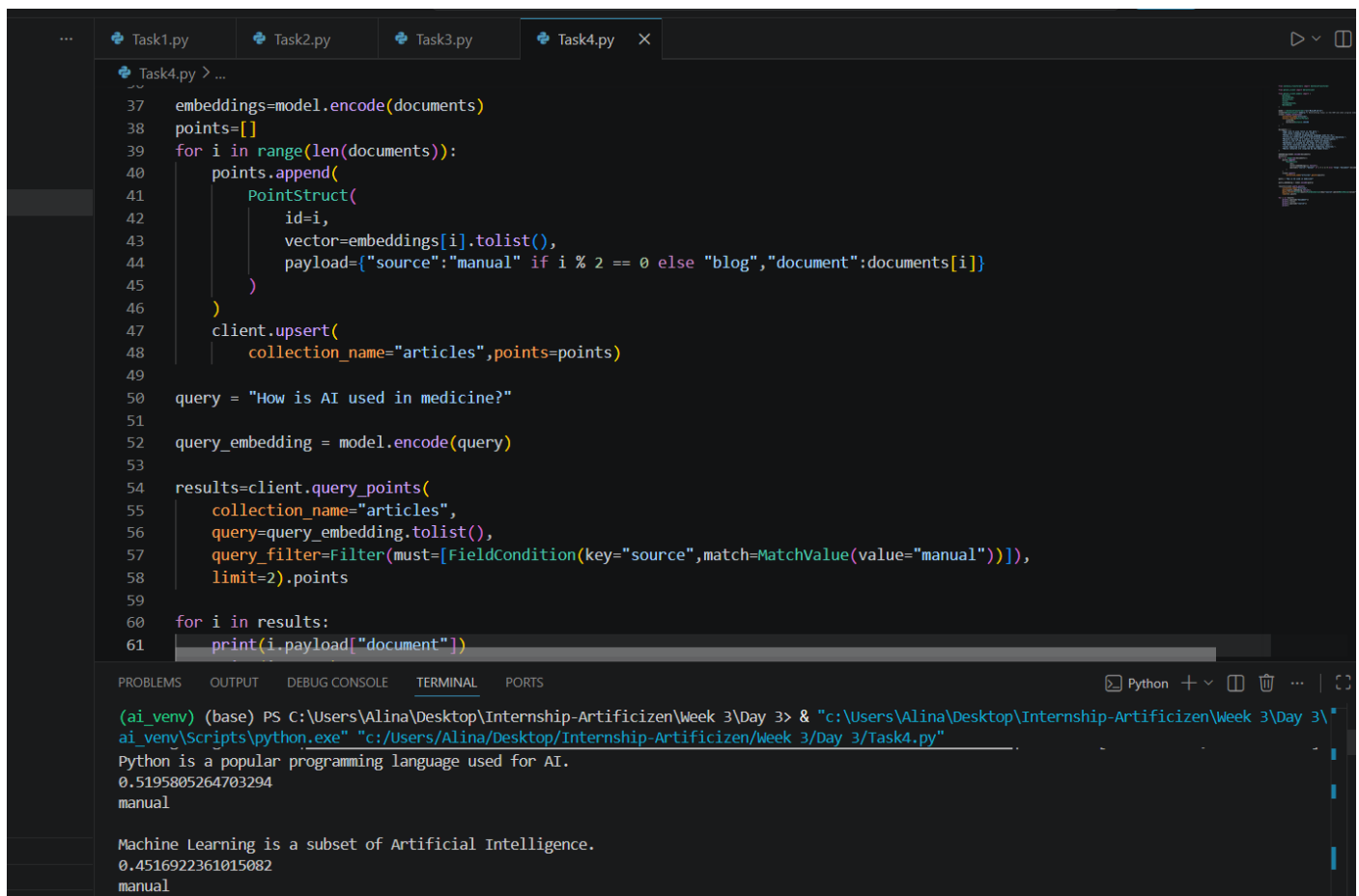
Terminal Output:

```
(ai_venv) (base) PS C:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 3> & "c:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 3\ai_venv\Scripts\python.exe" "c:/Users/Alina/Desktop/Internship-Artificizen/Week 3/Day 3/Task3.py"
Top 2 Similar Documents:
Document: Artificial Intelligence is transforming healthcare and education.
Cosine Similarity: 0.8453

Document: Python is a popular programming language used for AI.
Cosine Similarity: 0.9608

(ai_venv) (base) PS C:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 3>
```

4. Repeat the previous exercise using Qdrant (QdrantClient(':memory:')). Add payloads with a source field and filter results to only return documents where source == 'manual'.



The image shows a VS Code editor window with a file explorer at the top showing four files: Task1.py, Task2.py, Task3.py, and Task4.py. The Task4.py file is open and contains the following Python code:

```
37 embeddings=model.encode(documents)
38 points=[]
39 for i in range(len(documents)):
40     points.append(
41         PointStruct(
42             id=i,
43             vector=embeddings[i].tolist(),
44             payload={"source":"manual" if i % 2 == 0 else "blog","document":documents[i]}
45         )
46     )
47     client.upsert(
48         collection_name="articles",points=points)
49
50 query = "How is AI used in medicine?"
51
52 query_embedding = model.encode(query)
53
54 results=client.query_points(
55     collection_name="articles",
56     query=query_embedding.tolist(),
57     query_filter=Filter(must=[FieldCondition(key="source",match=MatchValue(value="manual"))]),
58     limit=2).points
59
60 for i in results:
61     print(i.payload["document"])
```

Below the code editor is a terminal window with the following output:

```
(ai_venv) (base) PS C:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 3> & "c:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 3\ai_venv\Scripts\python.exe" "c:/Users/Alina/Desktop/Internship-Artificizen/Week 3/Day 3/Task4.py"
Python is a popular programming language used for AI.
0.5195805264703294
manual

Machine Learning is a subset of Artificial Intelligence.
0.4516922361015082
manual
```

5. Embed 50 random sentences, then query with a sentence that is semantically close but uses completely different words. Verify that semantic search still finds the right match.

```
Task1.py Task2.py Task3.py Task4.py Task5.py X
Task5.py > ...
52     "The gardener watered the flowers.",
53     "The programmer fixed the bug.",
54     "The child is reading a storybook.",
55     "The astronaut explored space."
56 ]
57 embeddings=model.encode(documents)
58 def cosine_similarity(vec1,vec2):
59     return np.dot(vec1,vec2)/(np.linalg.norm(vec1)*np.linalg.norm(vec2))
60
61 query="How is AI used in medicine?"
62 query_embedding=model.encode(query)
63 results=[]
64 for i in range(len(documents)):
65     similarity=cosine_similarity(query_embedding,embeddings[i])
66     results.append((documents[i],similarity))
67
68 results.sort(key=lambda x:x[1],reverse=True)
69 print("Top 3 Similar Documents:")
70 for document, similarity in results[:3]:
71     print(f"Document: {document}")
72     print(f"Cosine Similarity: {similarity:.4f}")
73     print()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [] [X] ...

```
(ai_venv) (base) PS C:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 3> & "c:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day
ai_venv\Scripts\python.exe" "c:/Users/Alina/Desktop/Internship-Artificizen/Week 3/Day 3/Task5.py"
Top 3 Similar Documents:
Document: Artificial Intelligence is changing healthcare.
Cosine Similarity: 0.5855

Document: Doctors prescribe medicine.
Cosine Similarity: 0.4438

Document: Machine Learning uses data.
Cosine Similarity: 0.4169
```

6. Write a utility `embed_and_store(texts, metadata_list, collection)` that batch-embeds a list of strings and upserts them into a Qdrant collection with metadata. You will reuse this directly in the RAG pipeline tomorrow.

```
Task1.py Task2.py Task3.py Task4.py Task5.py Task6.py X
```

```
Task6.py > ...  
7     VectorParams,  
8     PointStruct  
9 )  
10  
11 model = SentenceTransformer("all-MiniLM-L6-v2")  
12 client=QdrantClient(":memory:")  
13 client.create_collection(  
14     collection_name="documents",  
15     vectors_config=VectorParams(  
16         size=384,  
17         distance=Distance.COSINE  
18     )  
19 )  
20  
21 def embed_and_store(texts,metadata_list,collection):  
22     embeddings = model.encode(texts)  
23     points=[]  
24     for i in range(len(texts)):  
25         points.append(  
26             PointStruct(  
27                 id=i,  
28                 vector=embeddings[i].tolist(),  
29                 payload=metadata_list[i]  
30             )  
31         )  
32     client.upsert(  
33         collection_name=collection,points=points)  
34     print("Documents stored successfully!")
```